

VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
 SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

IT149IU · FUNDAMENTALS OF PROGRAMMING (PYTHON)

Lab 1: Python Basics

Variables, operators, input/output, string formatting and simple decisions

Student name		Student ID	
Class / Lab group		Date	28/09/2026
Duration	150 minutes (3 periods)	Environment	Python 3.10+ · Colab / VS Code / PyCharm / Jupyter

Legend: black text = original lab content (re-formatted); red text = content added in this revision (theory, worked examples, extra exercises, setup & submission guide).

Course assessment (IT149IU – Fundamentals of Programming)

- **10 labs** in the semester.
- **Option 1:** mini-project(s) — students who choose this option do not take the final LAB exam (register after week 4).
- **Option 2:** no mini-project — take the final LAB exam.
- **Attendance – 30%:** check-in in class, MS Teams, and submission of all labs (via the course Google Form – see Section 6).
- **Lab reports – 40%:** 2–3 selected labs are scored.
- **Mini-project / final LAB exam – 30%.**

NOTE — Lab plan – 150 minutes (3 periods)

Time	Activity
0 – 20 min	Environment setup: Google Colab + one desktop IDE; Task 0 (Section 2)
20 – 40 min	Theory review and worked examples (Sections 3–4)
40 – 70 min	Exercises 1 – 4 (print, input, operators, eggs in boxes)
70 – 105 min	Exercises 5 – 9 (if/else, tables with f-strings, ord(), dictionaries)
105 – 135 min	Exercises 10 – 12 (time conversion, wage growth, sorting with if)
135 – 145 min	Challenge exercises 13 – 15 (optional, bonus)
145 – 150 min	Save, export and submit (Section 6)

1. Objectives

Goals: Introduce Python variables, operators, printing, and formatting using simple exercises.

After this lab you will be able to:

- Understand the difference between a variable (`x`) and a string (`'x'`).
- Learn how `print()` combines strings and variables.
- Use **f-strings** for cleaner formatting.
- Practice arithmetic operations inside `print()`.
- Read user input with `input()` and convert it with `int()` / `float()`.
- Use the operators `//`, `%` and `**`, and make decisions with `if` / `elif` / `else`.
- Set up and use Google Colab, VS Code, PyCharm, JupyterLab and Jupyter Notebook.

2. Environment Setup

In this course you may write and run Python in **five** environments. **Google Colab** needs no installation and is the recommended environment for the lab sessions; the four desktop IDEs (**Visual Studio Code**, **PyCharm**, **JupyterLab**, **Jupyter Notebook**) are used for larger programs and for working offline. Complete the setup once in this first lab — every later lab uses the same tools.

Environment	Type	Install?	Best for
Google Colab	Cloud notebook (browser)	No – only a Google account	Lab sessions, quick experiments, sharing
Visual Studio Code	Code editor + extensions	Yes	.py scripts, debugging, Git/GitHub
PyCharm (Community)	Full Python IDE	Yes	Larger projects, refactoring, debugging
JupyterLab	Notebook IDE (local browser)	Yes (pip)	Data analysis, several notebooks/files side by side
Jupyter Notebook	Classic notebook (local browser)	Yes (pip)	Simple notebooks, teaching/demos

2.1 Install Python 3 (for all desktop IDEs)

1. Go to <https://www.python.org/downloads/> and download the latest **Python 3.12 or 3.13** installer for your operating system.
2. **Windows:** run the installer and tick "**Add python.exe to PATH**" on the first screen, then click *Install Now*.
3. **macOS:** run the .pkg installer; afterwards run *Install Certificates.command* from the Python folder in *Applications*. (Alternative: `brew install python.`)
4. **Linux (Ubuntu/Debian):** Python 3 is pre-installed; add pip and venv with `sudo apt install python3-pip python3-venv`.
5. Open a terminal (Windows: *Command Prompt* or *PowerShell*; macOS/Linux: *Terminal*) and verify:

TERMINAL / COMMAND PROMPT [added]

```
python --version      # Windows -> Python 3.12.x
python3 --version     # macOS / Linux
python -m pip --version
```

TIP — Virtual environments (recommended)

A virtual environment keeps the packages of one project separate from other projects.

TERMINAL / COMMAND PROMPT [added]

```
python -m venv .venv          # create (once per project folder)
.venv\Scripts\activate       # Windows
source .venv/bin/activate     # macOS / Linux
pip install numpy pandas matplotlib seaborn jupyterlab notebook
```

2.2 Google Colab (no installation – recommended for labs)

Google Colaboratory runs Jupyter notebooks on Google's servers. You only need a web browser (Chrome, Edge, Firefox, Safari) and a Google account (your personal Gmail or your student Google account).

Step-by-step: getting started with Colab

1. Open <https://colab.research.google.com> and **sign in** with your Google account.
2. **Open this lab notebook:** *File* → *Upload notebook* → choose `FoP_Lab1.ipynb` (downloaded from the course page). Alternatively put the file in Google Drive, right-click it → *Open with* → *Google Colaboratory*. (If *Google Colaboratory* is not in the list: *Open with* → *Connect more apps* → search "Colaboratory" → *Install*.)
3. **Save your own copy immediately:** *File* → *Save a copy in Drive*. Your copy is stored in *My Drive/Colab Notebooks/*.
4. **Connect a runtime:** click *Connect* (top right). A green tick with RAM/Disk bars means a Python kernel is ready.
5. **Run a cell:** click inside a code cell and press **Shift + Enter** (or the  button). The output appears under the cell.
6. **Add cells:** *+ Code* and *+ Text* buttons (or `Ctrl+M B` for a new cell below). Text cells use Markdown.
7. **Install extra packages** (most scientific packages are pre-installed): `!pip install package_name` in a code cell.
8. **Use files:** click the  *Files* icon on the left → *Upload*, or mount Google Drive (code below). Uploaded files are deleted when the runtime is recycled — keep important files in Drive.
9. **Restart / reset:** *Runtime* → *Restart session* (keeps files) or *Runtime* → *Disconnect and delete runtime* (clean start). Use *Runtime* → *Run all* to check that the whole notebook works from top to bottom before submitting.
10. **Download / share:** *File* → *Download* → *Download .ipynb* (or *.py*); *Share* (top right) → *General access: Anyone with the link* – *Viewer* → *Copy link*. This link can be submitted in the Google Form.

PYTHON [added]

```
# Mount your Google Drive inside Colab (a pop-up asks for permission)
from google.colab import drive
drive.mount('/content/drive')
# Your files are now under /content/drive/MyDrive/
```

Shortcut (Colab)	Action
Shift + Enter	Run cell and move to the next cell
Ctrl + Enter	Run cell and stay
Ctrl + M, then B / A	Insert code cell below / above
Ctrl + M, then M / Y	Convert cell to text / code
Ctrl + M, then D	Delete cell
Ctrl + /	Comment / uncomment selected lines

COMMON MISTAKES — Colab limitations

- `input()` works in Colab (a text box appears under the cell), but GUI programs (Tkinter windows) cannot be displayed — run them in VS Code/PyCharm.
- Sessions disconnect after about 90 minutes of inactivity; variables are lost, so re-run the cells from the top.

2.3 Visual Studio Code

To run Python in Visual Studio Code, follow these steps:

- **Install Python:** Ensure Python is installed on your system (Section 2.1).
- **Install VS Code:** download from <https://code.visualstudio.com> and install it (Windows: tick "Add to PATH" and "Open with Code").
- **Install the Python Extension:** Open VS Code, go to the Extensions view (Ctrl+Shift+X), search for "Python" by Microsoft, and install it. Also install the **Jupyter** extension to open `.ipynb` notebooks directly in VS Code.
- **Select the interpreter:** Ctrl+Shift+P → *Python: Select Interpreter* → choose your Python 3.12 (or the `.venv` of the project).
- **Install Code Runner (Optional but Recommended):** In the Extensions view, search for "Code Runner" and install it. This extension provides a convenient "Run" button.
- **Open or Create a Python File:** Open the folder containing your Python files in VS Code (*File* → *Open Folder*), or create a new file with a `.py` extension.
- **Write Your Python Code:** Enter your Python code into the editor.
- **Run the Code:**
 - **Using Code Runner:** If you installed Code Runner, click the "Run Python File" button (a play icon) in the top right corner of the editor.

- **Using the Terminal:** Open the integrated terminal (`Ctrl` + backtick key) and navigate to the directory of your `.py` file. Then execute the file using `python your_file_name.py` (or `python3 your_file_name.py` depending on your Python installation).
- **Without Debugging:** Go to *Run* → *Run Without Debugging* or use the keyboard shortcut `Ctrl+F5`.
- **With Debugging:** Go to *Run* → *Start Debugging* or use the keyboard shortcut `F5`.

The output of your Python script will appear in the integrated terminal or the output panel within VS Code. If you are using Code Runner and need user input, ensure "Code Runner: Run in Terminal" is enabled in your VS Code settings.

2.4 PyCharm

Install: download *PyCharm Community Edition* (free) from <https://www.jetbrains.com/pycharm/download/> — or the unified *PyCharm* installer and choose the free tier. Students can also request a free Professional licence at <https://www.jetbrains.com/community/education/>. Create a project with *New Project* → *Pure Python*, keep "Create a virtualenv" selected and choose Python 3.12 as the base interpreter.

Running Python code in PyCharm can be accomplished through several methods, primarily involving Run Configurations.

1. Running a Python Script

- **Right-click in the editor:** open the Python file you want to run. Right-click anywhere within the editor and select *Run 'your_script_name'*. PyCharm will automatically create a temporary Run Configuration and execute the script, displaying the output in the Run tool window.
- **Green Play Button in Gutter:** locate the green "play" arrow icon in the gutter next to the `if __name__ == "__main__":` block or at the top of the file. Clicking this will run the current script.
- **Run Menu:** go to *Run* → *Run* in the main menu, or use the shortcut `Alt+Shift+F10` (Windows/Linux) or `Ctrl+Alt+R` (Mac). This brings up a menu of available Run Configurations; select the one corresponding to your script.
- **Run Configuration Dropdown:** at the top right of the PyCharm window there is a dropdown menu for Run Configurations. Select your desired configuration and click the green play button next to it.

2. Running a Selection in the Python Console

- **Select Code:** highlight the specific lines of Python code within the editor that you wish to execute.
- **Execute Selection:** right-click on the selected code and choose *Execute Selection in Python Console*, or use the shortcut `Alt+Shift+E`. This runs the selected code in the interactive Python console, allowing you to see immediate results and interact further.

3. Debugging

- **Set Breakpoints:** click in the gutter next to the line numbers to set breakpoints where you want the execution to pause.
- **Debug Mode:** use the "Debug" option instead of "Run" (right-click menu, Run menu, or the Debug button next to the Run Configuration dropdown). This launches the script in debug mode, allowing you to step through the code, inspect variables, and analyze program flow.

NOTE — Important considerations

- **Python Interpreter:** ensure that you have a Python interpreter configured for your project. PyCharm usually prompts you to set one up when creating a new project (*Settings → Project → Python Interpreter; install packages with the + button*).
- **Run Configurations:** for more complex projects or specific execution requirements, you can create and customize permanent Run Configurations via *Run → Edit Configurations...* This allows you to specify details like script path, working directory, environment variables, and interpreter options.

2.5 JupyterLab

JupyterLab is the modern notebook interface: a file browser, notebooks, terminals and text files in one browser tab.

1. Install (inside your virtual environment if you use one):

```
TERMINAL / COMMAND PROMPT [added]
```

```
pip install jupyterlab
jupyter lab           # opens http://localhost:8888/lab in your browser
```

2. In the *Launcher*, click **Python 3 (ipykernel)** under *Notebook* to create a notebook, or double-click an existing `.ipynb` in the left file browser (upload files with the `↑` button).
3. Run a cell with **Shift + Enter**; switch a cell to Markdown with `Esc` then `M`, back to code with `Esc` then `Y`.
4. *Kernel → Restart Kernel and Run All Cells...* checks that the notebook runs from a clean state.
5. *File → Save and Export Notebook As...* exports to `.py`, HTML or PDF (PDF needs LaTeX; use HTML then print to PDF instead).
6. Stop the server with `Ctrl+C` in the terminal (confirm with `y`).

2.6 Jupyter Notebook

The classic Jupyter Notebook (version 7) has a simpler, one-document-per-tab interface.

```
TERMINAL / COMMAND PROMPT [added]
```

```
pip install notebook
jupyter notebook      # opens http://localhost:8888/tree
```

1. The browser shows the *Files* dashboard of the folder where you started the command — start it from your course folder.
2. *New → Python 3 (ipykernel)* creates a notebook; click an `.ipynb` file to open it.
3. Rename the notebook by clicking its title (e.g. `Lab1_NguyenVanAn_ITITI24001`).
4. Run cells with **Shift + Enter**; *Kernel → Restart & Run All* before submitting.
5. *File → Download as → Notebook (.ipynb) / Python (.py) / HTML*.

TIP — Anaconda alternative

Installing **Anaconda Distribution** (<https://www.anaconda.com/download>) gives Python, JupyterLab, Jupyter Notebook, NumPy, pandas and Matplotlib in one installer; start everything from *Anaconda Navigator*.

2.7 Cursor (optional)

Cursor (<https://cursor.com>) is an AI-assisted editor based on VS Code; install it and configure Python exactly as in Section 2.3. **AI assistants may be used only as allowed by the course policy — declare their use in your submission.**

2.8 Task 0 – Check your environments

Example: print "Hello, *your full name*" and capture a screenshot **in at least two environments (Google Colab + one desktop IDE of your choice).**

PYTHON

```
full_name = "Nguyen Van An"      # <- replace with your full name
print(f"Hello, {full_name}")
```

PYTHON [added]

```
# Environment check – run in each environment and include the output in your report
import sys, platform
print("Python version :", sys.version.split()[0])
print("Executable      :", sys.executable)
print("Operating sys.  :", platform.system(), platform.release())
```

STUDENT WORK AREA — Task 0 – screenshots of "Hello, full name" in two environments

Paste your code and a screenshot of the output here.

3. Theory

In Python, variables are assigned using `=`. Strings are enclosed in quotes. The `print()` function displays output and can accept multiple arguments separated by commas. By default, `print()` separates arguments with a space and ends with a newline.

3.1 Variables and basic data types

A **variable** is a name that refers to a value. Python is *dynamically typed*: the type belongs to the value, not to the name, and `type()` tells you the type.

Type	Example	Meaning
int	x = 7	whole numbers (unlimited size)
float	price = 7.5	real numbers (about 15–16 significant digits)
str	name = 'Ada' or "Ada"	text (sequence of characters)
bool	ok = True	truth values True / False

Names may contain letters, digits and `_`, cannot start with a digit and are case-sensitive (`total` ≠ `Total`). Use descriptive *snake_case* names such as `eggs_per_box`.

PYTHON [added]

```
x = 2
y = 3.0
name = 'x'
print(type(x), type(y), type(name)) # <class 'int'> <class 'float'> <class 'str'>
print(x, name)                       # 2 x    -> variable value vs. the text 'x'
```

3.2 Arithmetic operators and precedence

Operator	Name	Example	Result
+	addition	7 + 2	9
-	subtraction	7 - 2	5
*	multiplication	7 * 2	14
/	true division (always float)	7 / 2	3.5
//	floor division (rounds down toward $-\infty$)	7 // 2, -7 // 2	3, -4
%	remainder (modulo)	7 % 2	1
**	exponentiation	2 ** 10	1024

Precedence (high → low): `()` → `**` → unary `-` → `*` / `/` / `//` / `%` → `+` / `-`. For example `-5 ** 2` is `-25` (the power is evaluated first) while `(-5) ** 2` is `25`. Use parentheses whenever in doubt.

3.3 Input and type conversion

`input(prompt)` always returns a **string**. Convert it before doing arithmetic: `int('5') → 5`, `float('2.5') → 2.5`, `str(42) → '42'`. Converting text that is not a number raises a `ValueError`.

3.4 Output: print() and f-strings

`print(*values, sep=' ', end='\n')` — `sep` is placed between the values, `end` after the last one. An **f-string** (`f"..."`) evaluates expressions inside `{ }` and can format them with a *format specifier* after a colon:

Specifier	Example	Output
:>8 right-align, width 8	f"{42:>8}"	42
:<8 left-align	f"{'ab':<8} "	ab
:^8 centre	f"{'ab':^8} "	ab

Specifier	Example	Output
:.2f 2 decimals	f"{3.14159:.2f}"	3.14
:, thousands separator	f"{1234567:,}"	1,234,567
:05d zero padding	f"{7:05d}"	00007
:.1% percentage	f"{0.256:.1%}"	25.6%

3.5 Decisions with if / elif / else

Comparison operators `==` `!=` `<` `<=` `>` `>=` produce `True` or `False`; combine conditions with `and`, `or`, `not`. The body of an `if` is the **indented** block (4 spaces) after the colon.

```
PYTHON [added]
n = 17
if n % 2 == 0:
    print(n, "is even")
elif n % 3 == 0:
    print(n, "is odd and divisible by 3")
else:
    print(n, "is odd")
```

4. Worked Examples

Run each example, predict the output **before** running it, then change the inputs and run again.

Worked Example 1 – Temperature converter (input, float, f-string).

```
PYTHON [added]
celsius = float(input("Temperature in °C: "))
fahrenheit = celsius * 9 / 5 + 32
print(f"{celsius:.1f} °C = {fahrenheit:.1f} °F")
```

Worked Example 2 – A formatted receipt (alignment and decimals).

```
PYTHON [added]
items = [("Notebook", 2, 12500), ("Pen", 5, 4000), ("Python book", 1, 185000)]
print(f"{'Item':<12}{'Qty':>5}{'Price':>10}{'Total':>12}")
grand = 0
for name, qty, price in items:
    total = qty * price
    grand += total
    print(f"{name:<12}{qty:>5}{price:>10}{total:>12},")
print(f"{'GRAND TOTAL':<27}{grand:>12} VND")
```

Worked Example 3 – Quotient and remainder. 125 minutes = 2 hours 5 minutes.

```
PYTHON [added]
minutes = 125
print(minutes // 60, "hours", minutes % 60, "minutes")
```

5. Exercises

Exercise 1 – Variables and print()

Create the variables `x = 2` and `y = 3`, then determine what each of the following statements displays:

- `print('x =', x)`
- `print('Value of', x, '+', x, 'is', (x + x))`
- `print('x =')`
- `print((x + y), '=', (y + x))`

Step-by-step

- Create variables:

PYTHON

```
x = 2
y = 3
```

- Statement a) — prints the string `'x ='` followed by the value of `x` (2). Result: `x = 2`

PYTHON

```
print('x =', x)
```

- Statement b) — combines text and variables; `(x + x) = 4`. Result: `Value of 2 + 2 is 4`

PYTHON

```
print('Value of', x, '+', x, 'is', (x + x))
```

- Statement c) — only prints the text, no variable. Result: `x =`

PYTHON

```
print('x =')
```

- Statement d) — `(x + y) = 5` and `(y + x) = 5`. Result: `5 = 5`

PYTHON

```
print((x + y), '=', (y + x))
```

Extended exercises

- Change `x = 7`, `y = 9` and re-run the code. Observe the output.
- Use `print` with the `sep` argument: `print('x=', x, sep=' ')` → prints `x=2` without a space.

- Use the end argument: `print('x=', x, end='; ')` followed by `print('y=', y)` → prints on the same line.
- Rewrite using f-strings: `print(f"x = {x}")`.

STUDENT WORK AREA — Exercise 1 – extended exercises (your code)

Paste your code and a screenshot of the output here.

SCHOLAR DISCUSSION

These exercises highlight how Python `print()` works and how arithmetic, variables, and strings are combined. Using f-strings is more modern and readable. Such basic understanding is foundational for writing clear outputs in data analysis and AI experiments. References: *Python Crash Course* (Eric Matthes), *Think Python* (Allen Downey).

Exercise 2 – Fixing the input type

The following code should calculate the square of a number:

PYTHON

```
number = input("Input a number between 1 and 10")
square = number * number
```

What's wrong with this code? Fix that code and write your code in a new code section.

What's wrong? In Python, `input()` always returns a string. Trying to multiply a string by a string is invalid. For example, input '5' → '5' (string), and '5' * '5' causes a `TypeError`.

Corrected code (integer):

PYTHON

```
number = int(input("Input a number between 1 and 10: "))
square = number * number
print("The square is:", square)
```

Alternative (float):

PYTHON

```
number = float(input("Enter any number: "))
square = number ** 2
print("Square:", square)
```

Extension

- Check the range [1, 10] and print an error if the number is out of range.
- Allow multiple numbers (loop from 1 to 5 and print the squares).

STUDENT WORK AREA — Exercise 2 – extension (your code)

Paste your code and a screenshot of the output here.

SCHOLAR DISCUSSION

This exercise illustrates the importance of data types. New programmers often forget that `input()` returns a string. Type casting ensures correct arithmetic operations. This concept connects to data preprocessing in DS/AI, where type conversion is frequent.

Exercise 3 – Arithmetic operators

Test each of the arithmetic operators `+`, `-`, `*`, `/`, `//` and `**` with -5, 0, 5 and 7.5 as the left operand and 2 as the right operand.

Quick reference: `+` addition · `-` subtraction · `*` multiplication · `/` true division (float) · `//` floor division (rounds down toward $-\infty$) · `**` exponent (power).

PYTHON

```
left_operands = [-5, 0, 5, 7.5]
right_operand = 2

for a in left_operands:
    print(f"\nTesting with left operand = {a}")
    print(a, "+", right_operand, "=", a + right_operand)
    print(a, "-", right_operand, "=", a - right_operand)
    print(a, "*", right_operand, "=", a * right_operand)
    print(a, "/", right_operand, "=", a / right_operand)
    print(a, "//", right_operand, "=", a // right_operand)
    print(a, "**", right_operand, "=", a ** right_operand)
```

Sample output & notes

- For a = -5: $-5 + 2 = -3$, $-5 - 2 = -7$, $-5 * 2 = -10$, $-5 / 2 = -2.5$, $-5 // 2 = -3$ (floor), and $a ** 2 = 25$ (because `a` holds -5; the literal expression `-5 ** 2` would give -25 – see Section 3.2)
- For a = 0: $0 + 2 = 2$, $0 - 2 = -2$, $0 * 2 = 0$, $0 / 2 = 0.0$, $0 // 2 = 0$, $0 ** 2 = 0$
- For a = 5: $5 + 2 = 7$, $5 - 2 = 3$, $5 * 2 = 10$, $5 / 2 = 2.5$, $5 // 2 = 2$, $5 ** 2 = 25$
- For a = 7.5: results are floats → 9.5, 5.5, 15.0, 3.75, 3.0, 56.25

Extensions

- Compare `/` vs `//` on negative numbers (e.g., $-5/2$ vs $-5//2$).
- Compute square roots via exponent: `a ** 0.5`.
- Format results with f-strings to align columns nicely.

STUDENT WORK AREA — Exercise 3 – extensions (your code)

Paste your code and a screenshot of the output here.

Exercise 4 – Eggs in boxes

Typically 6 eggs fit in one box. Write a script to calculate the number of boxes a farmer needs to store 28 eggs. The script will also determine how many eggs are placed in the last uncompleted box, and how many additional eggs are needed to fill this last box.

Concept

- Use floor division `//` to compute full boxes.
- Use modulo `%` to find the leftover eggs.
- If remainder $> 0 \rightarrow$ add 1 extra box.
- Eggs missing = `per_box - remainder`.

PYTHON

```
eggs = 28
per_box = 6

boxes = eggs // per_box + (1 if eggs % per_box else 0)
last_box_fill = eggs % per_box if eggs % per_box else per_box
need_to_fill_last = per_box - (eggs % per_box) if eggs % per_box else 0

print("Total eggs:", eggs)
print("Boxes needed:", boxes)
print("Eggs in last box:", last_box_fill)
print("Eggs needed to fill last box:", need_to_fill_last)
```

SAMPLE OUTPUT

```
Total eggs: 28
Boxes needed: 5
Eggs in last box: 4
Eggs needed to fill last box: 2
```

Extensions

- Allow user input for `eggs` and `per_box` to generalize.
- Draw an ASCII diagram showing boxes and eggs inside.

STUDENT WORK AREA — Exercise 4 – extensions (your code)

Paste your code and a screenshot of the output here.

Exercise 5 – Odd or even with the remainder operator

Use if statements to determine whether an integer is odd or even using the **remainder operator**.

Concept: if `number % 2 == 0` → the number is divisible by 2 → even; otherwise → odd.

PYTHON

```
number = int(input("Enter an integer: "))

if number % 2 == 0:
    print(number, "is even")
else:
    print(number, "is odd")
```

Sample runs: input 8 → 8 is even; input 13 → 13 is odd.

Extensions

- Use a for-loop to print the odd/even classification for the numbers 1–10.
- In pandas, filter DataFrame rows by even/odd index using `df.index % 2`.

STUDENT WORK AREA — Exercise 5 – extensions (your code)

Paste your code and a screenshot of the output here.

SCHOLAR DISCUSSION

Modulo-based conditions are a common programming pattern. In DS/AI preprocessing, conditional filtering (even/odd index, category group) is widely used. This simple concept scales to more complex logical conditions.

Exercise 6 – Bacteria growth table

Starting with 200 bacteria, the growth in the number of bacteria after n hours is calculated as follows: $B = 200 \times 2^n$. Print the number of bacteria after 0, 5, 10, and 15 hours in table format as shown below. Use the **tab escape sequence** (`\t`) to achieve the two-column output.

SAMPLE OUTPUT

Hour	Number of Bacteria
0	200
5	6400
10	204800
15	6553600

Concept: the number of bacteria doubles every hour (power of 2); use the exponentiation operator `**`; use `'\t'` (tab) to separate columns.

PYTHON

```
base = 200
hours = [0, 5, 10, 15]

print("Hour\tNumber of Bacteria")
for h in hours:
    B = base * (2 ** h)
    print(f"{h}\t{B}")
```

Extensions

- Allow the user to input a maximum hour and print the table for 0..max.
- Add a step size to control how often results are printed.
- Plot the bacteria growth curve using matplotlib.

STUDENT WORK AREA — Exercise 6 – extensions (your code)

Paste your code and a screenshot of the output here.

SCHOLAR DISCUSSION

This exercise illustrates exponential growth. In DS/AI, similar exponential functions appear in algorithms (time complexity, data growth) and in modeling natural processes. Visualizing such growth highlights the rapid scale of exponential functions.

Exercise 7 – Right-aligned bacteria growth table

Reimplement the code of Exercise 6 to produce the following table with the numbers **right aligned** in each column.

SAMPLE OUTPUT

Hour	Number of Bacteria
0	200
5	6400
10	204800
15	6553600

Hint: read <https://realpython.com/python-f-strings/> and <https://www.geeksforgeeks.org/string-alignment-in-python-f-string/>

Concept: f-strings support alignment with format specifiers: `:>5` right-aligns within 5 spaces, `:>12` within 12 spaces. The Hour column can be width 5 and the Number of Bacteria column width 12 (or width 20 to right-align the numbers under the whole header, as in the sample above).

PYTHON

```
base = 200
hours = [0, 5, 10, 15]

print(f'{"Hour":>5}\t{"Number of Bacteria":>12}')
for h in hours:
    B = base * (2 ** h)
    print(f'{h:>5}\t{B:>12}')
```

Extensions

- Try different widths (e.g., `>8` or `>15`).
- Add more hours dynamically and keep the alignment consistent.
- Export the table to CSV/Excel for analysis in pandas.

STUDENT WORK AREA — Exercise 7 – extensions (your code)

Paste your code and a screenshot of the output here.

SCHOLAR DISCUSSION

Proper alignment is important for readability in both console output and reports. In DS/AI, formatting tables helps present large datasets clearly. f-strings provide a clean, modern way to control layout compared to the older % formatting or `str.format()`.

Exercise 8 – Name score with `ord()`

Each of a string's **characters** has an integer representation. The set of characters a computer uses together with the characters' integer representations is called that computer's **character set**. You can indicate a character value in a program by enclosing that character in quotes, as in `"A"`. To determine a character's integer value, call the built-in function `ord` (e.g. `ord('A') → 65`). Tom and Jim want to play a game but are unable to choose who goes first. They decide that the person with the highest sum of the integer values of the characters of their name gets to play first. Write a script to determine who goes first.

Concept: `ord('A') = 65`, `ord('B') = 66`, etc. Sum `ord()` of all characters in a string to compute a name score, then compare the scores of Tom and Jim.

PYTHON

```
def name_score(name):
    return sum(ord(c) for c in name)

tom_score = name_score("Tom")
jim_score = name_score("Jim")
print("Tom score:", tom_score)
print("Jim score:", jim_score)

if tom_score > jim_score:
    print("Tom goes first!")
elif jim_score > tom_score:
    print("Jim goes first!")
else:
    print("It's a tie!")
```

SAMPLE OUTPUT

```
Tom score: 304
Jim score: 288
Tom goes first!
```

Correction: T = 84, o = 111, m = 109, so Tom's score is 304 (the previous version showed 296).

Extensions

- Allow user input of multiple names and compute their scores.
- Write a function `winner(names_list)` that finds the name with the highest score.
- Compare results across different names to observe how ASCII values affect outcomes.

STUDENT WORK AREA — Exercise 8 – extensions (your code)

Paste your code and a screenshot of the output here.

SCHOLAR DISCUSSION

This exercise demonstrates the use of ASCII/Unicode representations in computation. In DS/AI, character encoding is fundamental (e.g., tokenization, embeddings). Summing `ord()` values is a simple illustration of mapping categorical data to numeric features.

Exercise 9 – Course grades (average, best, worst)

Write a script that inputs the grades (in integers) of 3 courses from the user. First display the average grade of the three courses, and then the names and the grades of the courses with the highest and the lowest grades.

Concept

- Use a dictionary to map course names to grades.

- Average = `sum(grades) / number_of_courses`.
- Use `max(..., key=dict.get)` and `min(..., key=dict.get)` to find the highest/lowest.

Step-by-step guide

1. Input grades for three courses (e.g., Math, Physics, AI).
2. Store them in a dictionary mapping course name → grade.
3. Compute the average grade using `sum(values) / len(values)`.
4. Find the highest and lowest grades using `max()` and `min()` with `key=courses.get`.
5. Print the results clearly.

PYTHON

```
courses = {}
courses['Math'] = int(input('Enter grade for Math: '))
courses['Physics'] = int(input('Enter grade for Physics: '))
courses['AI'] = int(input('Enter grade for AI: '))

avg = sum(courses.values()) / len(courses)
best = max(courses, key=courses.get)
worst = min(courses, key=courses.get)

print('Average grade:', round(avg, 2))
print('Highest grade:', best, courses[best])
print('Lowest grade:', worst, courses[worst])
```

SAMPLE OUTPUT

```
Enter grade for Math: 85
Enter grade for Physics: 92
Enter grade for AI: 78
Average grade: 85.0
Highest grade: Physics 92
Lowest grade: AI 78
```

Extensions

- Allow any number of courses until the user types `'done'`.
- Save the results to CSV (course, grade) for later analysis.
- Use `pandas.Series(courses).describe()` to summarize statistics (mean, min, max, std).

STUDENT WORK AREA — Exercise 9 – extensions (your code)

Paste your code and a screenshot of the output here.

SCHOLAR DISCUSSION

This exercise shows how to collect structured input and perform descriptive statistics. In Data Science, we often collect attributes (courses/grades) and then compute summary metrics. Python dictionaries and pandas provide powerful tools for such tasks.

Exercise 10 – Seconds to hours – minutes – seconds

Write a script that inputs a number of seconds from the user. Calculate the number of hours, minutes, and remaining seconds. Print them separated by " - ". For example, if the user types 3750 seconds as input, the script should print:

SAMPLE OUTPUT

1 - 2 - 30

Assume that the user enters a number of seconds that is **higher than 3600**. Use both the floor division and the remainder operation to calculate the number of hours, minutes, and seconds.

Revision note: the previous version of this handout repeated the Exercise 9 solution under this heading; the guidance below matches the actual task.

Concept: 1 hour = 3600 s and 1 minute = 60 s. "Peel off" the hours with `// 3600`, keep the rest with `% 3600`, then split the rest into minutes and seconds.

PSEUDOCODE [added]

```
READ total_seconds
hours      <- total_seconds // 3600
remainder <- total_seconds % 3600
minutes    <- remainder // 60
seconds    <- remainder % 60
PRINT hours, " - ", minutes, " - ", seconds
```

STARTER CODE (PYTHON) [added]

```
total_seconds = int(input("Enter a number of seconds (> 3600): "))
hours = total_seconds // 3600
# TODO: compute minutes and seconds using // and %
minutes = ...
seconds = ...
print(hours, minutes, seconds, sep=" - ")
```

Extensions: validate that the input is > 3600; also print the time as HH:MM:SS with `f"{hours:02d}:{minutes:02d}:{seconds:02d}"`; convert back and check that `hours*3600 + minutes*60 + seconds == total_seconds`.

STUDENT WORK AREA — Exercise 10 – your complete solution

Paste your code and a screenshot of the output here.

Exercise 11 – Employee wage growth / decline

Every year, if an employee receives a good job performance review, they get a raise of 3% on their wages. In turn, if they receive a suboptimal performance review, their wages are deducted by 3%. Consider an employee starting with an hourly wage of \$10. Calculate how much this employee is earning after 5 years of consistent good reviews and after 2 years of bad reviews. Use the following formula:

$w = o(1 + p)^n$, where w is the new hourly wage, o is the original hourly wage, p is the percentage increase or decrease, and n is the number of years with an increase or decrease in hourly wage.

Concept: the annual raise/cut is compounded multiplicatively; use the exponent operator `**` for powers, `(1 + p) ** n`; round the final wage to 2 decimals when printing.

PYTHON

```
o = 10.0                # original hourly wage ($/hour)

p_good, n_good = 0.03, 5
w_good = o * (1 + p_good) ** n_good
print('After 5 years good reviews:', round(w_good, 2))

p_bad, n_bad = -0.03, 2
w_bad = o * (1 + p_bad) ** n_bad
print('After 2 years bad reviews:', round(w_bad, 2))
```

SAMPLE OUTPUT

```
After 5 years good reviews: 11.59
After 2 years bad reviews: 9.41
```

Extensions

- Let the user input o , p , and n ; compute the final wage.
- Accept a sequence of reviews like `['good', 'bad', 'good']` and apply 1.03 or 0.97 each year.
- Plot wage-by-year using matplotlib to visualize the compounding effect.

STUDENT WORK AREA — Exercise 11 – extensions (your code)

Paste your code and a screenshot of the output here.

SCHOLAR DISCUSSION

This task models compound growth/decay, a core mathematical pattern also used in finance and in learning-rate schedules in ML. Multiplicative compounding differs from linear growth: small percentage changes accumulate over time.

Exercise 12 – Sorting runner times with if only

After a running competition, three runners want to determine who won based on the time they needed to complete the course. Input three floating-point times (seconds) for the three runners and display them in increasing order. Only use plain `if` statements (no `else`, no built-in `sort`).

PYTHON

```
a = float(input('Enter time for runner 1: '))
b = float(input('Enter time for runner 2: '))
c = float(input('Enter time for runner 3: '))

print('Times in ascending order:')
if a <= b and b <= c: print(a, b, c)
if a <= c and c <= b: print(a, c, b)
if b <= a and a <= c: print(b, a, c)
if b <= c and c <= a: print(b, c, a)
if c <= a and a <= b: print(c, a, b)
if c <= b and b <= a: print(c, b, a)
```

Notes: this approach tests all 6 possible permutations. It works but may print duplicates if values are equal. It demonstrates exhaustive branching with `if` conditions.

Extension: remove the duplicate-printing problem by using strict comparisons for some of the conditions (e.g. `a <= b and b < c`) or a variable `printed = False`; then print which runner (1, 2 or 3) won.

STUDENT WORK AREA — Exercise 12 – extension (your code)

Paste your code and a screenshot of the output here.

Challenge Exercises (optional, bonus)

Exercise 13 – BMI calculator

Input a person's weight (kg) and height (m), compute $BMI = \text{weight} / \text{height}^2$, and print it with one decimal together with the category: *Underweight* (< 18.5), *Normal* ($18.5 - < 25$), *Overweight* ($25 - < 30$), *Obese* (≥ 30).

SAMPLE OUTPUT [added]

```
Weight (kg): 68
Height (m): 1.72
BMI = 23.0 -> Normal
```

STUDENT WORK AREA — Exercise 13 – your code

Paste your code and a screenshot of the output here.

Exercise 14 – Cash withdrawal (//, %)

An ATM only has 500,000 / 200,000 / 100,000 / 50,000 VND notes. Input an amount that is a multiple of 50,000 and print the smallest number of notes of each type (hint: repeat the "quotient and remainder" idea of Exercise 10).

SAMPLE OUTPUT [added]

```
Amount (VND): 1850000
500,000 x 3
200,000 x 1
100,000 x 1
50,000 x 1
```

STUDENT WORK AREA — Exercise 14 – your code

Paste your code and a screenshot of the output here.

Exercise 15 – Digits of a number

Input a three-digit integer (e.g. 472). Using only `//` and `%`, print its hundreds, tens and units digits, the sum of the digits, and the reversed number (274). Print whether the number is a *palindrome* (e.g. 353).

STUDENT WORK AREA — Exercise 15 – your code

Paste your code and a screenshot of the output here.

6. Submission Instructions

What to submit

Each student must submit the following:

1. **Source code (.py or .ipynb):**
 - Include all exercises (1–12 **and any challenge exercises you solved**).
 - Add your name, student ID, and comments at the top of each file, e.g. `Lab1_LyTuNga_ID_ex1.py`.
2. **Screenshots of outputs:**
 - Capture console or Jupyter outputs for each exercise.
 - Make sure the results match the expected outputs.
3. **Lab report (.docx or .pdf):** include Goals, Theory, Objectives, Step-by-step explanation, Sample outputs, Extended exercises, and Scholar discussion.
4. **Optional (for extra credit):** a Matplotlib chart from Exercise 6 or a pandas summary from Exercise 9.

5. **Submission format:** file name `IT149IU_Lab1_FullName_StudentID.zip` (or a folder / GitHub repository with the same name).

```
IT149IU_Lab1_FullName_StudentID/
├── Lab1_Report.docx
├── exercises/
│   ├── ex1_print.py
│   ├── ex2_input.py
│   ├── ex3_arithmetic.py
│   └── ...
├── screenshots/
│   ├── ex1_output.png
│   ├── ex2_output.png
│   └── ...
```

6. **Upload method:** submit the link of your folder/repository through the course **Google Form** (below) – this replaces the previous Blackboard upload. Deadline: end of the same lab week (Sunday 23:59).

How to submit (Google Form)

All labs are submitted through **one shared Google Form**. Submit a **link** to a folder or repository that contains your work.

Step 1 – Organize your work in one folder named `IT149IU_Lab1_FullName_StudentID` (e.g. `IT149IU_Lab1_NguyenVanAn_IT149IU24001`):

```
IT149IU_Lab1_FullName_StudentID/
├── FoP_Lab1_FullName_StudentID.ipynb
├── codes/
│   ├── Task1.py
│   └── ...
├── outputs/
│   ├── task1_output.png
│   └── ...
├── FoP_Lab1_Report.pdf
└── README.md
```

- the **notebook** – completed, with all cells executed;
- **codes/** – one `.py` file per task, each starting with the header comment;
- **outputs/** – screenshots that clearly show input and output;
- the **lab report** (`.docx` or `.pdf`) and a **README.md** (name, student ID, how to run, notes).

Step 2 – Upload the folder using ONE of these options:

1. **GitHub (recommended):** create a repository (e.g. `IT149IU-FoP-Labs`, one sub-folder per lab), push your files and make the repository *Public* (or add the instructor as a collaborator).
2. **Google Drive:** upload the folder → right-click → *Share* → *General access: Anyone with the link – Viewer* → *Copy link*.
3. **Google Colab only:** *Share* → *Anyone with the link – Viewer* → *Copy link* (use this only if all work is inside the notebook).

TERMINAL / COMMAND PROMPT [added]

```
# GitHub from the command line (first time)
git init
git add .
git commit -m "Lab 1 - FullName - StudentID"
git branch -M main
git remote add origin https://github.com/<your-username>/IT149IU-FoP-Labs.git
git push -u origin main
# later labs: git add . ; git commit -m "Lab N" ; git push
```

Step 3 – Submit the link in the Google Form

Item	Details
Submission form for Lab 1 (Lab number pre-selected)	Open the Lab 1 submission form
Short link (choose "Lab 1" in the form)	https://forms.gle/aHqDLDZ6YUyzoYVv8
Fields to fill in	Full name · Student ID · Class/Group · Lab number · Submission type · Submission link
Deadline	End of the lab week – Sunday 23:59 (unless the instructor announces another date)

Before you press Submit – checklist

- The notebook runs from top to bottom without errors (*Runtime* → *Restart session and run all*).
- Every task has code, output (screenshot or cell output) and, where requested, pseudocode.
- Each `.py` file starts with the header comment (name, ID, course, lab, task, date, description).
- The link opens in a private/incognito browser window (sharing permission is correct).
- You may re-submit (edit your response) before the deadline; the last submission is graded.

NOTE — Grading guide (per lab)

Criterion	Weight
Correctness of programs (all required tasks)	50%
Pseudocode / algorithm and explanation	15%
Code quality (names, comments, functions, input validation)	15%
Output evidence (screenshots / executed notebook)	10%
Extensions and challenge tasks	10%

COMMON MISTAKES — Academic integrity

Discuss ideas with classmates, but write your own code. Copied submissions receive zero for all students involved. If you use an AI assistant, state which tool and for which part in the *Notes* field of the form and in your README.